Explicación Detallada del Código - Ruffcol E-commerce Backend

Este documento detalla el propósito, funcionamiento y diseño de cada componente del código fuente del proyecto Ruffcol (Backend en Spring Boot y Frontend Vanilla).

1. Configuración (`com.ruffcol.tienda.config`)

- **SecurityConfig.java**: Configura Spring Security, define qué rutas son públicas (como autenticación y catálogo) y cuáles requieren autenticación o roles específicos (ADMIN, SUPER_ADMIN). Desactiva CSRF por ser una API REST y configura el filtro JWT.
- **JWTUtil.java**: Gestiona la creación, firma (con HS256) y validación de los tokens JSON Web Tokens (JWT), extrayendo el ID, nombre de usuario y roles con una vigencia de 7 días.
- **JWTAuthenticationFilter.java**: Intercepta cada solicitud HTTP entrante para verificar la presencia del encabezado `Authorization: Bearer <token>`, validarlo y establecer la sesión en el contexto de seguridad.
- **FileUploadConfig.java**: Administra los parámetros para la recepción y almacenamiento de imágenes de productos (límite de tamaño de 10MB).
- **OpenApiConfig.java**: Configura Swagger UI para la documentación interactiva de los endpoints REST en `/swagger-ui.html`.

2. Controladores REST (`com.ruffcol.tienda.controlador`)

- **AuthController.java**: Maneja los puntos de acceso públicos `/api/auth/register-cliente` y `/api/auth/login`, procesando credenciales y retornando tokens JWT cifrados con BCrypt.
- **CategoriaController.java**: Expone los endpoints CRUD para las categorías de productos, restringiendo las operaciones de escritura (crear, actualizar, eliminar) a administradores y validando que no existan productos asociados al eliminar.
- **ProductoController.java**: Gestiona la consulta pública de productos y el filtrado por categoría, así como su administración (CRUD completo con soporte de imágenes).
- **PedidoController.java**: Permite a los clientes autenticados crear pedidos a partir de los elementos del carrito (actualizando automáticamente el stock) y consultar sus propios pedidos o los de todo el sistema por parte de los administradores.
- **SuperAdminController.java**: Ofrece endpoints exclusivos para el rol `SUPER_ADMIN` orientados a la creación y listado de administradores.

3. Capa de Servicios (`com.ruffcol.tienda.servicio`)

Contiene la lógica de negocio pura:

- **UsuarioServiceImpl.java**: Lógica de registro con encriptación de contraseñas (BCrypt) y validación de unicidad.
- **ProductoServiceImpl.java**: Operaciones de negocio para la gestión de inventario y asociación de categorías.
- **CategoriaServiceImpl.java**: Validaciones de nombres únicos y control de integridad referencial.
- **PedidoServiceImpl.java**: Manejo transaccional para la creación de pedidos, cálculo de totales y descuento de existencias de stock.
- **FileStorageService.java**: Lógica de guardado y eliminación física de archivos en el sistema de almacenamiento local.

4. Repositorios (`com.ruffcol.tienda.repositorio`)

Interfaces basadas en **Spring Data JPA** que extienden `JpaRepository` (`UsuarioRepository`, `ProductoRepository`, `CategoriaRepository`, `PedidoRepository`, `DetallePedidoRepository`), proporcionando operaciones CRUD automáticas y consultas personalizadas por atributos clave.

5. Modelos y DTOs (`com.ruffcol.tienda.modelo` y `dto`)

- **Modelos (Entidades JPA)**: Mapean las tablas relacionales de la base de datos H2 (`Usuario`, `Producto`, `Categoria`, `Pedido`, `DetallePedido`) junto con sus enumeraciones (`Rol`, `EstadoCuenta`, `EstadoPedido`).
- **DTOs (Data Transfer Objects)**: Encapsulan y transfieren datos específicos entre el frontend y el backend, evitando exponer información confidencial como contraseñas hashes.